

Jayawant Shikshan Prasarak Mandal

CS3204L-B App development using Swift

CS3204L-B Programming Lab II

(App development using Swift programming)

Laboratory Manual

Computer Engineering

CS2211L Programming Lab I

(Swift programming)

Laboratory Manual

Computer Engineering

Author : - Dr. Rushali A. Deshmukh

Creation Date: - June 2025

Version : - 1

© JSPM Group of Institutes, Pune. All Rights Reserved. All the information in this Course Manual is confidential. Participants shall refrain from copying, distributing, misusing or disclosing the content to any third parties any circumstances whatsoever.

Table of Contents

Sr . No.	Topic	Page. No.
A.	**Vision, Mission**	4
B.	**PEOs and Pos**	5
C.	**PSOs**	7
D.	**Course Objectives**	8
E.	**Lab Plan**	
F. List of Experiments		
1.	Create a new project with label and a greet function.	9
2.	Create a project to use Label(UILabel),Image view, Text view, Scroll view	14
3.	Create a project to useTable view, Toolbars(UIToolbar), Navigation bars, tab bars.	18
4.	Create a project to use Sliders, Switches, Date pickers	23
5.	Create a new project called Calculator using the iOS App template.	27
6.	Create a guided project Apple Pie	32
7.	Implement Playgrounds for Optionals, Type casting and Inspection, Guard, Constant and variable scope, Enumerations	35
8.	Create a new project called "Login" using the iOS App template.	39
9.	Create a new project called "AboutMe" using the iOS App template. Use tab bar controller.	43
10.	Create a new project called "OrderOfEvents" using the iOS App template.	47
G.	**References**	51

Vision of Department

To create quality computer professionals through an excellent academic environment.

Mission of Department

1. To empower students with the fundamentals of Computer Engineering for being successful professionals.
2. To motivate the students for higher studies, research, and entrepreneurship by imparting quality education.
3. To create social awareness among the students.

Program Educational Objectives: -

PEO I **Graduate shall have successful professional careers, lead and manage teams.**

PEO II **Graduate shall exhibit disciplinary skills to resolve real life problems.**

PEO III **Graduate shall evolve as professionals or researchers and continue to learn emerging technologies.**

Program Outcomes: -

Engineering Graduates will be able to:

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSO) addressed by the Course:

A graduate of the Computer Engineering Program will demonstrate-

PSO1: Technical Proficiency and Domain Specialization -

Graduates will be able to understand, analyze, and develop computer-based systems using domain-specific knowledge in areas such as algorithms, system software, web development, data science, artificial intelligence, computer networks and security.

PSO2: Problem Solving Skills - Graduates will be able to demonstrate strong problem-solving abilities by analyzing practical issues, formulating efficient solutions, and develop robust software systems using modern tools and technologies and software engineering methodologies.

PSO3:Empowered Career Development- Graduates will be able to demonstrate professional competence, entrepreneurial mindset, and a commitment to lifelong learning to excel in industry, research, entrepreneurship, and interdisciplinary fields.

CO	Description
CO1:	Apply Xcode, MVC pattern, and Interface Builder to develop simple iOS applications with interactive user interface components. BL3
CO2:	Analyze UIKit components, Auto Layout, and stack views to design adaptive and user-friendly iOS application interfaces. BL4
CO3:	Evaluate the use of Swift constructs and navigation workflows to manage application logic and enhance user experience. BL5
CO4:	Design and build complete iOS applications by integrating user interface design, navigation controllers, auto layout, and app life cycle management. BL6

Sub code Subject	CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO12	PSO1	PSO2	PSO3
CS3204L-B Programming Lab II (App development using Swift programming)	CO1	3	2	2		3				1	2		2	3	2
	CO2	3	3	2	2	3				2	2		2	3	3
	CO3	2	3	2	2	3			1	2	3		2	2	3
	CO4	3	3	3	2	3			1	3	3	2	3	3	3
	Avg.	2.75	2.75	2.25	2	3			1	2	2.5	2	2.25	2.75	2.75

EXPERIMENT NO.1

Create a new project with Label and a Greet function.

TITLE: Creating a simple iOS project with UILabel and a greeting function.

OBJECTIVES:

1. To understand how to create a new iOS project in Xcode.
2. To learn how to add and configure UILabel in the storyboard.
3. To implement a function in Swift that updates label text dynamically.
4. To connect UI components to code using IBOutlet and IBAction.
5. To develop a basic interactive application in Swift.

PROBLEM STATEMENT:

Design an iOS application that displays a label with default text and changes the text to a greeting message when a function is triggered by a button press.

SOFTWARE REQUIRED:

1. Xcode (latest stable version)
2. Swift (comes with Xcode)
3. iOS Simulator / iPhone device

INPUT:

User taps a button to call the greet function.

OUTPUT:

The label updates to display “Hello, Welcome to Swift Programming!”

THEORY:

iOS development with Swift involves both UI design (visual elements) and logic implementation (code). In this experiment, we work with the most fundamental UI element: the UILabel.

UILabel

UILabel is a UI component that displays static or dynamic text on the screen.

It is part of the UIKit framework.

Properties of UILabel include:

text: The content of the label.

textColor: Sets the color of the text.

font: Defines font style and size.

textAlignment: Defines alignment (left, center, right).

Example:

myLabel.text = "Hello World!"

myLabel.textColor = UIColor.blue

myLabel.font = UIFont.systemFont(ofSize: 20)

Creating a New Project in Xcode

Open Xcode → Select New Project.

Choose the template: App (iOS).

Enter product name (e.g., “GreetApp”).

Select interface: Storyboard, language: Swift.

Run the project in simulator → default screen appears.

Adding UILabel and UIButton in Storyboard

Open Main.storyboard.

Drag and drop a UILabel and a UIButton from the object library.

Position them properly in the view.

Change the button title to “Greet”.

Connecting UI to Code

Open the Assistant editor → Control-drag the label to ViewController.swift → Create an IBOutlet.

@IBOutlet weak var myLabel: UILabel!

Control-drag the button → Create an IBAction.

```
@IBAction func greetUser(_ sender: UIButton) {
    myLabel.text = "Hello, Welcome to Swift Programming!"
}
```

Function in Swift

A function is a reusable block of code.

Syntax:

```
func greet() -> String {
```

```
    return "Hello!"
}
```

In iOS apps, functions are often linked to UI events like button taps.

Event Handling

The @IBAction is triggered when the user interacts with a UI element.

This connects the visual element (UIButton) with the logic (Swift code).

App Lifecycle Context

When the app is launched, viewDidLoad() runs first.

The user interacts with the button → triggers greetUser() → updates label.

This experiment demonstrates the core workflow of iOS app development:

Create UI elements (UILabel, UIButton).

Connect them to Swift code.

Write logic in a function to update the UI.

CONCLUSION:

The experiment successfully created an iOS application that uses UILabel and a greeting function. The student learned project setup, label properties, and how to connect UI components to Swift code.

OUTCOME:

Students are able to create new iOS projects in Xcode.

Students can use UILabel and UIButton effectively.

Students can connect UI with Swift code using IBOutlets and IBActions.

Students understand basic function implementation in Swift.

SHORT QUESTIONS & ANSWERS

Q: What is UILabel used for in iOS?
A: UILabel is used to display text (static or dynamic) on the screen.

Q: How do you connect a label from storyboard to code?
A: Using IBOutlet.

Q: What is the purpose of IBAction?
A: To connect a UI element (like a button) to a function that responds to user actions.

Q: Which method is called first when a view is loaded?
A: viewDidLoad().

Q: Can UILabel display multiple lines of text? If yes, how?
A: Yes, by setting numberOfLines = 0 and enabling word wrapping.

EXPERIMENT NO.2

Create a project to use Label (UILabel), Image View, Text View, Scroll View

TITLE: Building an iOS app with UILabel, UIImageView, UITextView, and UIScrollView.

OBJECTIVES:

To learn how to integrate multiple UI components in a single iOS screen.

To understand the role of UILabel, UIImageView, UITextView, and UIScrollView.

To demonstrate content display with scrolling functionality.

To implement a layout that combines text and images.

PROBLEM STATEMENT

Design an iOS application that displays a label, an image, and descriptive text inside a scroll view so that the user can scroll through the content.

SOFTWARE REQUIRED:

Xcode (latest version)

Swift programming language (bundled with Xcode)

iOS Simulator or iPhone device

INPUT:

User scrolls through the content.

OUTPUT:

The app shows a label at the top, an image below it, and a long text that can be scrolled vertically.

THEORY:

This experiment introduces four important UIKit components:

UILabel

Displays text on the screen.

Can be used for headings, captions, or messages.

Common properties:

text → The displayed string.

font → Font type and size.

textColor → Sets the text color.

numberOfLines → Allows multi-line text if set to 0.

Example:

titleLabel.text = "Welcome to Swift App"

titleLabel.font = UIFont.boldSystemFont(ofSize: 24)

UIImageView

Displays images in the app.

Supports PNG, JPEG, and system images (SF Symbols).

Can be set from assets folder or directly in code.

Example:

imageView.image = UIImage(named: "swift_logo")

UITextView

A scrollable text container that can hold long paragraphs.

Editable or non-editable depending on the requirement.

Supports formatting (bold, italic, colored text).

Common properties:

text → The displayed text.

isEditable → Controls whether users can modify text.

font → Font style.

Example:

descriptionTextView.text = "Swift is a powerful programming language..."

descriptionTextView.isEditable = false

UIScrollView

A container that enables scrolling when content exceeds screen size.

Works with both horizontal and vertical scrolling.

Requires content size to be set larger than the frame.

Used when combining images, text, or large layouts.

Example:

scrollView.contentSize = CGSize(width: view.frame.width, height: 1200)

Steps to Create the App

Create a new iOS project in Xcode → App template.

Open Main.storyboard.

Drag and drop a UIScrollView into the View Controller.

Place a UILabel, UIImageView, and UITextView inside the Scroll View.

Configure constraints so that they are vertically aligned.

Add sample text to the Text View (long paragraph) and sample image in the Assets.

Connect the UI elements to ViewController.swift using IBOutlet.

Run the app → The label, image, and text appear and can be scrolled.

CONCLUSION:

We successfully created an iOS project that demonstrates the use of UILabel, UIImageView, UITextView, and UIScrollView. The user can view text and images by scrolling through the screen.

OUTCOME:

Students can use UILabel for headings.

Students can display images using UIImageView.

Students can present large text content with UITextView.

Students understand how UIScrollView enables navigation through large content.

SHORT QUESTIONS & ANSWERS:

Q: What is the main difference between UILabel and UITextView?
A: UILabel is for short static text, while UITextView is for multi-line, scrollable, and optionally editable text.

Q: How do you add images in an iOS app?
A: By adding images to the Assets catalog and using UIImageView.

Q: Can UIScrollView scroll horizontally?
A: Yes, by setting its content width larger than the screen width.

Q: What property of UITextView allows editing?
A: The isEditable property.

Q: Why do we need UIScrollView when using UITextView?
A: UITextView scrolls text internally, but UIScrollView allows multiple UI elements (text + image + labels) to scroll together.

EXPERIMENT NO.3

Create a project to use Table View, Toolbars, Navigation, Tab Bars

TITLE: Building an iOS app using UITableView, UIToolbar, UINavigationController, and UITabBarController.

OBJECTIVES:

1. To learn how to implement Table Views for displaying lists of data.
2. To understand the use of Navigation Controllers for hierarchical navigation.
3. To use Toolbars for quick actions.
4. To explore Tab Bars for multi-screen navigation.

PROBLEM STATEMENT:

Design an iOS app that displays a list of items using a Table View, allows navigation to detail screens, provides action buttons via a Toolbar, and supports multiple tabs using a Tab Bar.

SOFTWARE REQUIRED:

Xcode (latest version)

Swift programming language

iOS Simulator or real device

INPUT:

User taps on Table View items.

User navigates using Navigation Bar and Toolbar buttons.

User switches between screens using Tab Bar.

OUTPUT:

A working iOS application where:

Table View displays a list of elements.

Selecting an item navigates to a detail screen.

Toolbar shows actions (like Edit, Add).

Tab Bar switches between different sections of the app.

THEORY:

1. UITableView (Table View)

A scrollable list used to display multiple rows of data.

Each row is a UITableViewCell.

Can display simple text or complex custom views.

Common methods:

numberOfRowsInSection → Defines how many rows.

cellForRowAt → Configures each row's content.

didSelectRowAt → Handles tap action.

Example:

```
func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
    return items.count
}
func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
UITableViewCell {
    let cell = tableView.dequeueReusableCell(withIdentifier: "cell", for: indexPath)
    cell.textLabel?.text = items[indexPath.row]
    return cell
}
```

2. UINavigationController (Navigation Controller)

Provides hierarchical navigation.

Displays a Navigation Bar at the top with title and back button.

Supports push and pop of view controllers.

Example:

```
navigationController?.pushViewController(detailVC, animated: true)
```

3. UIToolbar (Toolbar)

Appears at the bottom of the screen.

Contains action buttons like Add, Edit, Delete.

Can be configured programmatically or via Storyboard.

Example:

```
let addButton = UIBarButtonItem(barButtonSystemItem: .add, target: self, action:
#selector(addItem))
toolbarItems = [addButton]
```

4. UITabBarController (Tab Bar)

Provides multiple sections in the app.

Each tab represents a different view controller.

Users can switch screens easily.

Example:

let tabBarVC = UITabBarController()

tabBarVC.viewControllers = [homeVC, settingsVC]

STEPS TO CREATE THE APP

Create a new iOS project in Xcode.

In Storyboard, embed the main View Controller in a Navigation Controller.

Add a Table View and set its prototype cell identifier to "cell".

Implement the Table View data source and delegate methods in ViewController.swift.

Add a Toolbar at the bottom and configure action buttons.

Create a Detail View Controller for Table item selection and connect via segue.

Add a Tab Bar Controller and connect it with two different View Controllers (e.g., Home & Settings).

Run the app → User can navigate between items, use Toolbar, and switch tabs.

CONCLUSION:

We successfully created an iOS project that demonstrates the use of Table View, Navigation, Toolbar, and Tab Bar. The user can browse lists, view details, perform actions, and navigate between app sections.

OUTCOME:

Students can design apps with list-based UI using Table Views.

Students understand navigation using UINavigationController.

Students can add Toolbar buttons for quick actions.

Students can design multi-tabbed applications using UITabBarController.

SHORT QUESTIONS & ANSWERS:

Q: What is the difference between Table View and Collection View?

A: Table View displays items in a single column, whereas Collection View supports multiple rows and columns (grid layout).

Q: Why is Navigation Controller used in iOS apps?

A: It helps in hierarchical navigation, allowing push and pop of view controllers.

Q: Can we add multiple buttons in a Toolbar?

A: Yes, Toolbars support multiple UIBarButtonItems.

Q: What is the role of a Tab Bar Controller?

A: It allows switching between multiple sections of an app.

Q: Which method is used to handle row selection in a Table View?

A: didSelectRowAt method of UITableViewDelegate.

EXPERIMENT NO.4

Create a project to use Sliders, Switches, and Date Pickers.

TITLE: Implementation of Sliders, Switches, and Date Pickers in Swift

OBJECTIVES:

To understand and implement UIKit controls: UISlider, UISwitch, and UIDatePicker.

To respond to user input through actions and events.

To learn how to display dynamic feedback from UI controls.

PROBLEM STATEMENT:

Design an iOS app where users can adjust a slider, toggle a switch, and select a date using a Date Picker. The app should display real-time feedback for each control.

SOFTWARE REQUIRED:

Xcode IDE (latest version)

Swift programming language

iOS Simulator or physical device

INPUT:

Slider value (continuous number range)

Switch state (ON/OFF)

Date selected via Date Picker

OUTPUT:

Label updates for slider value.

Label updates for switch state.

Label shows selected date in readable format.

THEORY:

UISlider:

A UISlider allows selection of a continuous value between a minimum and maximum.

Can be used for volume, brightness, or any adjustable parameter.

Event triggered: valueChanged → calls IBAction method.

Example properties:

minimumValue

maximumValue

value → current slider value

@IBAction func sliderChanged(_ sender: UISlider) {

```
    sliderValueLabel.text = "Value: \(Int(sender.value))"
}
```

UISwitch:

UISwitch is a toggle button with ON and OFF states.

Can be used for enabling/disabling features.

Event triggered: valueChanged → calls IBAction.

Example:

```
@IBAction func switchChanged(_ sender: UISwitch) {
    switchLabel.text = sender.isOn ? "Switch is ON" : "Switch is OFF"
}
```

UIDatePicker:

UIDatePicker allows users to select date, time, or both.

Can be displayed as wheels or compact style.

Event triggered: valueChanged → calls IBAction.

Date can be formatted using DateFormatter.

```
@IBAction func datePickerChanged(_ sender: UIDatePicker) {
    let formatter = DateFormatter()
    formatter.dateStyle = .medium
    formatter.timeStyle = .short
    dateLabel.text = formatter.string(from: sender.date)
}
```

Use Cases:

Slider → Adjust settings like volume.

Switch → Enable dark mode.

Date Picker → Select birth date or appointment.

CONCLUSION:

This experiment demonstrates how to use interactive controls to capture user input and provide real-time feedback.

OUTCOME:

Students will be able to:

Implement UISlider, UISwitch, and UIDatePicker.

Handle UI events to update views dynamically.

SHORT QUESTIONS & ANSWERS:

Q. What property of UISlider determines its range?

A: minimumValue and maximumValue.

Q.Which method is called when a switch is toggled?

A: valueChanged → linked via IBAction.

Q.How can we format the date from UIDatePicker?

A: Using DateFormatter.

Q.Can UISlider have integer values only?

A: No, but we can cast the Float value to Int.

Q.What is the default state of a UISwitch?

A: OFF (isOn = false).

EXPERIMENT NO.5

Create a new project called Calculator using the iOS App template

TITLE:
Create a new project called Calculator using the iOS App template with Auto Layout

OBJECTIVES:

1. To design a functional Calculator application in iOS.
2. To learn Auto Layout for arranging UI elements like buttons and labels.
3. To handle multiple UIButton actions and perform arithmetic operations.
4. To develop an interactive GUI similar to the iPhone calculator.

PROBLEM STATEMENT:

Develop a calculator app where the user can input numbers, perform arithmetic operations (Addition, Subtraction, Multiplication, Division), and view results. The buttons and display should be properly aligned using Auto Layout to mimic the standard iPhone calculator layout.

SOFTWARE REQUIRED:

Xcode IDE (latest version)

Swift Programming Language

iOS Simulator or iPhone device

INPUT:

User taps number buttons (0–9)

User taps operation buttons (+, −, ×, ÷)

User taps equals (=) button to compute result

Optional: Clear (C) button to reset

OUTPUT:

Computed result displayed in a UILabel at the top of the screen

Real-time display of numbers as they are typed

THEORY:

iOS UI Components:

UILabel: Displays numbers and results.

UIButton: Represents digits (0–9) and arithmetic operations.

UIView: Contains all buttons and acts as a layout container.

Auto Layout:

Auto Layout ensures consistent placement and sizing of UI elements across different device sizes.

Constraints define how buttons and labels relate to each other and to the parent view.

Common constraints:

Leading / Trailing / Top / Bottom

Width / Height

Aspect Ratio

Using stack views is recommended for arranging rows of buttons, similar to the iPhone calculator.

Calculator Logic:

Variables: Store current input, previous input, selected operation, and result.

Operations: Addition (+), Subtraction (−), Multiplication (×), Division (÷)

Steps for calculation:

User taps a number → append to current input

User taps an operator → store current input and operation

User taps another number → store as second input

User taps equals (=) → perform operation and display result

Swift Code Example:

```
var firstNumber: Double = 0
var secondNumber: Double = 0
var operation: String = ""
var isTypingNumber = false
@IBAction func numberPressed(_ sender: UIButton) {
    let number = sender.currentTitle!
    if isTypingNumber {
        displayLabel.text! += number
    } else {
        displayLabel.text = number
```

```
        isTypingNumber = true
    } }
@IBAction func operationPressed(_ sender: UIButton) {
    operation = sender.currentTitle!
    firstNumber = Double(displayLabel.text!)!
    isTypingNumber = false
}
@IBAction func equalsPressed(_ sender: UIButton) {
    secondNumber = Double(displayLabel.text!)!
    var result: Double = 0
    switch operation {
    case "+": result = firstNumber + secondNumber
    case "-": result = firstNumber - secondNumber
    case "×": result = firstNumber * secondNumber
    case "÷": result = secondNumber != 0 ? firstNumber / secondNumber : 0
    default: break
    }
    displayLabel.text = "\(result)"
    isTypingNumber = false
}
@IBAction func clearPressed(_ sender: UIButton) {
    displayLabel.text = "0"
    firstNumber = 0
    secondNumber = 0
    operation = ""
    isTypingNumber = false
}
```

Stack Views for Layout:

Create vertical stack view containing horizontal stack views for each row of buttons.

Buttons should have equal width and height using equal width/height constraints.

Align the display UILabel above the button stack with constraints to leading, trailing, and top margins.

Best Practices:

Use IBOutlet collections for buttons to handle actions efficiently.

Use padding and spacing in stack views for iPhone-like appearance.

Test on different iPhone simulators to ensure proper layout.

CONCLUSION:

This experiment demonstrates how to create a functional calculator in Swift with a professional iPhone-style GUI using Auto Layout. Students learn to combine UI design with logic programming.

OUTCOME:

Students will be able to:

Implement a calculator application in iOS using Swift.

Use Auto Layout to arrange buttons and labels in a consistent, responsive layout.

Handle multiple UIButton actions and perform arithmetic operations.

Design GUI similar to standard iPhone calculator.

SHORT QUESTIONS AND ANSWERS:

Q: What is the purpose of Auto Layout in iOS?
A: Ensures UI elements are properly positioned and sized across different screen sizes.

Q: How do you handle multiple buttons with a single IBAction?
A: Using IBOutlet collections or sender.currentTitle.

Q: How do you prevent division by zero?
A: Check if the second number is zero before performing division.

Q: What Swift variable stores the current operation in the calculator?
A: A String variable (e.g., operation).

Q: Why use stack views in this experiment?
A: Stack views simplify layout of multiple buttons and maintain equal spacing and sizing.

EXPERIMENT NO.6

Create a guided project Apple Pie

TITLE:
Developing a Word Guessing Game (Apple Pie) in Swift

OBJECTIVES:

To create a word-guessing game in iOS.

To learn the use of labels, buttons, and arrays for game logic.

To practice implementing conditional statements and loops.

PROBLEM STATEMENT:

Design a game where a player guesses a hidden word by selecting letters. Each wrong guess reduces the number of apples (lives). Correct letters are revealed in their positions.

SOFTWARE REQUIRED:

Xcode IDE

Swift programming language

INPUT:

Letter selections by the user using UIButton controls.

OUTPUT:

Label displays guessed letters and underscores for missing letters.

Visual indicator (apples) reduces on incorrect guesses.

THEORY:

Game Logic:

Store a list of words.

Randomly select one word for each game session.

Track letters guessed and remaining attempts.

UI Components:

UILabel → Display progress of the word (underscores and guessed letters).

UIButton → Each alphabet button for guessing.

UIImageView → Display remaining apples.

Conditional Logic:

If guessed letter is correct, update the word label.

If guessed letter is wrong, decrement remaining lives and update image.

Swift Constructs Used:

Arrays → Store words and guessed letters.

Loops → Iterate over letters of the word.

Functions → Handle guess evaluation and UI update.

Player wins if all letters guessed correctly.

Player loses if all apples are gone.

CONCLUSION:

Students successfully developed an interactive word game demonstrating Swift logic, UI components, and event handling.

OUTCOME:

Students will be able to:

Implement game logic in Swift.

Use multiple UI components and events.

Apply arrays, loops, and conditional statements in an iOS app.

SHORT QUESTIONS & ANSWERS:

Q.How is a random word selected from a list?
A: Using Int.random(in:) with the array index.

Q.What Swift control is used for guessing letters?
A: UIButton.

Q.How is the game progress displayed?
A: Using a UILabel with underscores for unguessed letters.

Q.How are incorrect guesses tracked?
A: Using a counter and updating an image of apples.

Q.How does the game determine a win?
A: When all letters of the word are correctly guessed.

EXPERIMENT NO.7

Implement Playgrounds for Optionals, Type Casting and Inspection, Guard, Constant and Variable Scope, Enumerations

TITLE: Implementing Swift Language Concepts in Playgrounds

OBJECTIVES:

1. To understand the concept of Optionals in Swift.
2. To explore type casting and type inspection.
3. To apply the use of guard statements for safe code execution.
4. To distinguish between constant and variable scope.
5. To implement enumerations in Swift for structured data representation.

PROBLEM STATEMENT:

Develop a Swift Playground that demonstrates Optionals, Type Casting, Guard statements, Constants and Variables scope, and Enumerations with working code examples.

SOFTWARE REQUIRED:

Xcode IDE

Swift Playground environment

INPUT:

Sample code snippets demonstrating Optionals, Type Casting, Guard, Scope, and Enumerations.

OUTPUT:

Execution results showing safe unwrapping of Optionals, type safety, guard-based validation, scope restrictions, and enum-based decisions.

THEORY:

Swift is a type-safe and expressive programming language that provides advanced language constructs to ensure safe coding practices.

Optionals:

An Optional in Swift is a variable that can hold either a value or nil.

Defined using ? (Optional type).

Optionals help in handling the absence of values safely.

Unwrapping is required to access the value using either forced unwrapping ! or safe unwrapping with if let or guard let.

Example:

var name: String? = "Alice"

```
if let actualName = name {
    print("Hello, \(actualName)")
}
```

Type Casting and Inspection:

Swift supports is (type check) and as (type casting).

as? is used for safe casting, returning an Optional.

Helps when working with class hierarchies and different data types.

Example:

```
let number: Any = 5
if let intNumber = number as? Int {
    print("Casted to Int: \(intNumber)")
}
```

Guard Statements:

Used for early exit if a condition fails.

Improves code readability and avoids deep nesting.

Example:

```
func validate(age: Int?) {
    guard let userAge = age, userAge >= 18 else {
        print("Not eligible")
        return
    }
    print("Eligible with age \(userAge)")
}
```

Constant and Variable Scope:

Constants are declared using let and cannot change.

Variables are declared using var.

Both follow scope rules (local, global, block-level).

Enumerations (Enums):

Provide a way to define a group of related values.

Can include associated values and methods.

Example:

```
enum Direction {
    case north, south, east, west
}
let move = Direction.north
```

These features are crucial for building safe, maintainable, and structured Swift applications.

CONCLUSION:

The experiment demonstrates core Swift concepts including Optionals, type casting, guard statements, scope management, and enumerations, which are essential for building reliable Swift programs.

OUTCOME:

Students will be able to write safe Swift code using Optionals, apply type casting, enforce program correctness using guard statements, understand scope, and implement enumerations for structured programming.

SHORT QUESTIONS AND ANSWERS:

Q: What is an Optional in Swift?

A: An Optional is a type that can either hold a value or nil.

Q: What is the difference between as? and as! in type casting?

A: as? safely attempts casting and returns an Optional, while as! forcefully unwraps and may cause runtime error if casting fails.

Q: Why is guard preferred over if let in some cases?

A: guard allows early exit, making the code cleaner and reducing nesting.

Q: What is the difference between let and var in Swift?

A: let declares a constant (immutable), while var declares a variable (mutable).

Q: Can enums in Swift have associated values?

A: Yes, enums can store associated values and even define methods.

EXPERIMENT NO.8

Create a new project called "Login" using the iOS App template.

TITLE: Design and Implement a Login Application in Swift

OBJECTIVES:

1. To develop a simple login application using Swift and Xcode.
2. To understand UI components such as Text Fields, Labels, and Buttons.
3. To implement event handling and validation logic for login.
4. To practice MVC (Model-View-Controller) pattern in iOS development.

PROBLEM STATEMENT:
Create a login app where the user enters a username and password. If the credentials match predefined values, display a success message; otherwise, show an error.

SOFTWARE REQUIRED:

Xcode IDE

Swift language support

INPUT:
Username and Password entered by the user.

OUTPUT:
Success message for valid credentials, error message for invalid credentials.

THEORY:
A Login Application is one of the most common iOS applications. It demonstrates the interaction between user interface elements and business logic.

User Interface (UI) Components:

UILabel: Displays static text or messages.

UITextField: Allows the user to enter input (username, password).

UIButton: Triggers actions when tapped (Login button).

Event Handling in Swift (IBAction):

Swift provides target-action mechanism.

An @IBAction method is linked with the UI button in the storyboard.

Example:

```
@IBAction func loginTapped(_ sender: UIButton) {
    // code to validate credentials
}
```

Logic Implementation:

Credentials can be predefined for demonstration.

Use conditional statements (if-else) for validation.

Example:

let validUsername = "admin"let validPassword = "1234"

MVC Design Pattern:

Model: Data (username and password).

View: Storyboard UI elements.

Controller: Handles logic and updates UI.

Storyboard Setup:

Place two UITextFields for username and password.

Place a UIButton labeled “Login”.

Place a UILabel for showing the result message.

SAMPLE CODE SNIPPET:

```
import UIKit
class ViewController: UIViewController {

    @IBOutlet weak var usernameField: UITextField!
    @IBOutlet weak var passwordField: UITextField!
    @IBOutlet weak var resultLabel: UILabel!

    @IBAction func loginTapped(_ sender: UIButton) {
        let username = usernameField.text
        let password = passwordField.text

        if username == "admin" && password == "1234" {
```

```
            resultLabel.text = "Login Successful ✓"
            resultLabel.textColor = .green
        } else {
            resultLabel.text = "Invalid Credentials ✗"
            resultLabel.textColor = .red
        }
    }
}
```

CONCLUSION:
The login application demonstrates how to integrate UI components, event handling, and logic in Swift using Xcode. It highlights the MVC architecture for iOS apps.

OUTCOME:
Students will be able to create interactive applications with user input, validate credentials, and display results based on conditions.

SHORT QUESTIONS AND ANSWERS:

Q: Which UI components are used for creating a login screen?
A: UITextField, UILabel, and UIButton.

Q: What is the purpose of @IBAction in Swift?
A: It is used to connect a UI event (e.g., button click) to a method in the controller.

Q: What design pattern does iOS follow for app development?
A: Model-View-Controller (MVC).

Q: How can you change the color of text in a UILabel?
A: By setting the textColor property, e.g., label.textColor = .red.

Q: Can we store multiple users' login details directly in code?
A: For learning purposes yes, but in real apps credentials should be securely stored in databases or keychains.

EXPERIMENT NO. 9

Create a new project called *"AboutMe"* using the iOS App template. Use Tab Bar Controller.

TITLE: Create a new project called “AboutMe” using the iOS App template. Use Tab Bar Controller.

OBJECTIVES:

To learn the use of Tab Bar Controller in iOS application development.

To create a simple project that displays personal information across multiple tabs.

To understand how multiple view controllers are managed using a tab bar.

To practice navigation and UI design in Swift.

PROBLEM STATEMENT:

Design an iOS application named “AboutMe” where multiple tabs are used to showcase different information about the user such as personal details, hobbies, and contact information using the Tab Bar Controller.

SOFTWARE REQUIRED:

Xcode IDE (latest version)

Swift programming language

iOS Simulator / iPhone device

INPUT:

User taps different tabs on the tab bar to view different sections (e.g., Profile, Hobbies, Contact).

OUTPUT:

The application displays different screens based on the selected tab:

Profile tab shows user’s basic details.

Hobbies tab lists hobbies/interests.

Contact tab shows contact details.

THEORY:

The Tab Bar Controller is one of the most commonly used navigation controllers in iOS applications. It allows users to switch between different sections of an app quickly by tapping icons at the bottom of the screen.

Each tab represents a UIViewController, which can have its own UI elements and logic. The tab bar is highly customizable and can display icons, titles, or both for each tab.

In this experiment, the AboutMe app will use a Tab Bar Controller with three tabs:

Profile Tab – Displays personal information such as name, age, and a short description.

Hobbies Tab – Shows hobbies or favorite activities using a text view or image view.

Contact Tab – Displays contact details like email or phone number.

Steps to implement:

Open Xcode and create a new iOS App project named AboutMe.

Select Storyboard Interface and Swift as the language.

In Main.storyboard, set a Tab Bar Controller as the initial view controller.

Add three view controllers connected to the Tab Bar Controller.

Assign each view controller a different title and icon (e.g., Profile, Hobbies, Contact).

Add UILabels, UITextViews, or UIImageViews inside each view to display relevant information.

Run the project on a simulator to test switching between tabs.

Tab Bar Controllers are widely used in apps like Instagram (Home, Search, Reels, Profile), WhatsApp (Chats, Status, Calls), and many more. They enhance user experience by organizing information into categories accessible through a bottom bar.

CONCLUSION:

By completing this experiment, we successfully developed an iOS application using the Tab Bar Controller to navigate between different views. The experiment demonstrates how information can be logically categorized and presented in an organized manner.

OUTCOME:

Students will be able to:

Implement a Tab Bar Controller in iOS apps.

Manage multiple UIViewControllers under one navigation system.

Build a multi-tab application that enhances usability and navigation.

SHORT QUESTIONS AND ANSWERS:

Q: What is the purpose of a Tab Bar Controller in iOS?
A: It allows switching between different view controllers using a tab bar at the bottom of the screen.

Q: Can we use images as tab bar icons?
A: Yes, custom images can be set as tab bar icons for each tab.

Q: How do you connect a view controller to a Tab Bar Controller?
A: By embedding the view controller in the Tab Bar Controller or connecting it via storyboard segues.

Q: What is the difference between a Tab Bar Controller and a Navigation Controller?
A: Tab Bar Controller allows switching between multiple sections, while Navigation Controller manages hierarchical navigation within a section.

Q: Can a Tab Bar Controller contain a Navigation Controller?
A: Yes, each tab can itself embed a Navigation Controller to manage deeper navigation.

EXPERIMENT NO. 10

Create a new project called *"OrderOfEvents"* using the iOS App template. Use Tab Bar Controller.

TITLE: Create a new project called “OrderOfEvents” using the iOS App template. Use Tab Bar Controller.

OBJECTIVES:

1. To study the view lifecycle methods in iOS applications.
2. To create a project that demonstrates the order of execution of various lifecycle events.
3. To use a Tab Bar Controller for switching between multiple view controllers and observing lifecycle behavior.

PROBLEM STATEMENT:

Design an iOS application named “OrderOfEvents” that uses a Tab Bar Controller to navigate between different screens. Each screen should display logs or labels showing which lifecycle methods are called (such as viewDidLoad(), viewWillAppear(), viewDidAppear(), etc.), so that students can understand the order of execution of events.

SOFTWARE REQUIRED:

Xcode IDE (latest version)

Swift programming language

iOS Simulator / iPhone device

INPUT:

User switches between tabs on the tab bar.

OUTPUT:

Application displays messages (via print() logs in console or UILabels on the screen) showing the order in which view controller lifecycle methods are triggered.

THEORY:

In iOS, each UIViewController follows a lifecycle that defines the order of events from creation to destruction. The most commonly used lifecycle methods are:

viewDidLoad() – Called once when the view is first loaded into memory. Used for initial setup.

viewWillAppear() – Called every time before the view becomes visible.

viewDidAppear() – Called when the view is fully visible to the user.

viewWillDisappear() – Called just before the view is removed or hidden.

viewDidDisappear() – Called after the view is removed from the screen.

By using a Tab Bar Controller, students can switch between different view controllers, and observe the sequence of lifecycle methods called during transitions.

Steps to implement:

Open Xcode and create a new iOS App project named OrderOfEvents.

In Main.storyboard, set a Tab Bar Controller as the initial view controller.

Add multiple view controllers connected to the Tab Bar Controller.

In each view controller's Swift file, override lifecycle methods (viewDidLoad(), viewWillAppear(), etc.).

Add print() statements or UILabels inside each method to track when they are executed.

Run the app and switch between tabs to observe lifecycle event order in the debug console or on-screen.

This experiment helps visualize the lifecycle flow of an iOS application and clarifies how controllers behave when users navigate through different tabs.

CONCLUSION:

By completing this experiment, we successfully developed an iOS application that uses Tab Bar Controller to demonstrate the order of lifecycle events in view controllers. Students gained practical understanding of how and when lifecycle methods are triggered in iOS apps.

OUTCOME:

Students will be able to:

Implement Tab Bar Controller in an iOS project.

Override and observe UIViewController lifecycle methods.

Understand the execution order of lifecycle events during navigation.

SHORT QUESTIONS AND ANSWERS:

Q: What is the purpose of viewDidLoad() in iOS?
A: It is called once when the view is loaded into memory, and is typically used for initial setup.

Q: When is viewWillAppear() called?
A: Just before the view is presented on the screen.

Q: How is viewDidAppear() different from viewWillAppear()?
A: viewWillAppear() is called before the view is shown, while viewDidAppear() is called after the view is fully visible.

Q: Why do we use print statements in this experiment?
A: To track the order of lifecycle events as they are executed.

Q: Can lifecycle methods be used for data fetching?
A: Yes, typically lightweight setup is done in viewWillAppear(), while heavy operations are better suited in viewDidLoad().

REFERENCES

T1. Matthew Mathias, John Gallagher, Swift Programming: The Big Nerd Ranch Guide 2nd edition, 2015.
T2. Matt Neuberg , iOS 12 Programming Fundamentals with Swift, OReilly; 5th edition.
T3. Developing Swift Fundamentals (Xcode 13)